

IBVAP — Comprehensive SIH Judge Q&A Repository

17. 50 Detailed Questions & Answers

Category 1: Computer Vision & AI (Q1 – Q10)

Q1: Which YOLO version does IBVAP use, and why?

- **Fast Answer:** We use YOLO11 Small (`yolo11s.pt`) and Nano (`yolo11n.pt`) via the Ultralytics API.
- **Detailed Answer:** YOLO11 provides an optimal trade-off between inference speed and detection accuracy. It features enhanced feature extraction backbone blocks (C3k2) and SPPF, making it fast enough for real-time edge processing while accurately detecting human targets (`class 0`).

Q2: How does ByteTrack differ from SORT or DeepSORT?

- **Fast Answer:** ByteTrack retains low-confidence detections instead of throwing them away.
- **Detailed Answer:** Standard SORT/DeepSORT discard bounding boxes with confidence below a fixed threshold (e.g. 0.5), losing tracks during partial occlusion. ByteTrack uses a two-stage association algorithm matching high-confidence boxes first, then matching remaining tracks against low-confidence boxes ($0.10 \leq \text{conf} < 0.25$), drastically reducing ID switches.

Q3: What is the exact confidence threshold set in your tracking configuration?

- **Fast Answer:** `track_high_thresh` is 0.25, `track_low_thresh` is 0.10, and `new_track_thresh` is 0.30.
- **Detailed Answer:** Configured in `bytetrack_day3.yaml`, high-confidence detections (≥ 0.25) initiate track matching, low-confidence detections ($0.10-0.25$) recover occluded tracks, and new tracks require a score of ≥ 0.30 .

Q4: How long does ByteTrack maintain an unassigned track before deleting it?

- **Fast Answer:** 60 frames (`track_buffer: 60`).
- **Detailed Answer:** In `bytetrack_day3.yaml`, `track_buffer` is set to 60 frames (approx. 2 seconds at 30 FPS), allowing the system to keep a target's Track ID active even if they are temporarily hidden behind an obstacle.

Q5: What is your model's exact mAP (mean Average Precision)?

- **Fast Answer:** Model accuracy was not experimentally measured on a custom benchmark in the current codebase.
- **Detailed Answer:** We follow a zero-hallucination principle: while standard YOLO11 pretrained on COCO achieves state-of-the-art mAP on human classes, specific dataset mAP was not evaluated in our prototype files.

Q6: How do you handle detection of non-human objects like animals or vehicles?

- **Fast Answer:** We filter detections explicitly using `classes=[0]`.
- **Detailed Answer:** In `src/main.py` (Line 20), `model.track()` is passed `classes=[0]`, restricting detections exclusively to COCO class 0 (`person`), preventing stray animals or moving military vehicles from triggering false

intrusion events.

Q7: Can IBVAP detect intrusions at night or in foggy weather?

- **Fast Answer:** Yes, by swapping RGB weights for fine-tuned Thermal/Infrared (FLIR) YOLO11 weights.
- **Detailed Answer:** The underlying pipeline (YOLO11 + ByteTrack + Polygon Zone) is domain-agnostic. In thermal IR feeds, human targets appear as bright heat signatures, which YOLO11 detects with high accuracy.

Q8: How do you define the restricted zone coordinates?

- **Fast Answer:** Using a 2D polygon vertex array.
- **Detailed Answer:** A polygon $P = \{(x_1, y_1), (x_2, y_2), \dots\}$ defines the boundary. The centroid of each tracked bounding box is evaluated against the polygon using point-in-polygon spatial testing.

Q9: What happens if two intruders cross paths and overlap (occlusion)?

- **Fast Answer:** ByteTrack's Kalman filter predicts position while IoU matching resolves overlap.
- **Detailed Answer:** The Kalman filter maintains velocity vectors for both targets. During overlap, low-confidence score fusion maintains track continuity until targets separate.

Q10: What resolution does your video processing pipeline support?

- **Fast Answer:** Tested on 720p and 1080p surveillance video feeds (`data/video.mp4`).

Category 2: Database & Architecture (Q11 – Q20)

Q11: Why did you choose MongoDB Atlas for event storage?

- **Fast Answer:** For schemaless JSON flexibility and cloud availability.
- **Detailed Answer:** Intrusion events contain varying metadata (Track ID, frame index, risk, evidence file URLs). MongoDB stores natively formatted JSON documents without requiring SQL schema migrations.

Q12: What happens if the internet connection to MongoDB fails?

- **Fast Answer:** IBVAP automatically falls back to local CSV logging (`intrusion_log.csv`).
- **Detailed Answer:** Our 3-tier data service (`src/app.py` Lines 356–380) attempts MongoDB first. If offline, it reads and writes to `output/intrusion_log.csv`. If CSV is missing, it loads seed data. The system never crashes.

Q13: How does local CSV data get uploaded to MongoDB when internet returns?

- **Fast Answer:** `save_events_to_mongodb()` upserts CSV records using `update_one` with `upsert=True`.
- **Detailed Answer:** Whenever local CSV events are loaded, the system automatically attempts to sync them up to MongoDB Atlas using `event_id` as the unique matching key.

Q14: Where are the database secrets stored?

- **Fast Answer:** In environment variables or Streamlit Cloud Secrets.
- **Detailed Answer:** Never in source code. `get_mongo_uri()` checks `os.environ` keys (`MONGODB_URI`) or `st.secrets["MONGODB_URI"]`.

Q15: What is the database collection structure?

- **Fast Answer:** Database **IBVAP**, collection **intrusion_events**.

Q16: How do you ensure unique IDs across events?

- **Fast Answer:** Standard sequential keying (**EVT-001**, **EVT-002**, **EVT-003**).

Q17: Can IBVAP connect to Supabase instead of MongoDB?

- **Fast Answer:** Yes, **streamlit_app/utils/data_service.py** includes a built-in Supabase client loader.

Q18: What is stored in **intrusion_log.csv versus **secure_audit_log.csv**?**

- **Fast Answer:** **intrusion_log.csv** contains basic event columns; **secure_audit_log.csv** includes SHA-256 hashes (**previous_hash**, **record_hash**).

Q19: Is database query speed cached in Streamlit?

- **Fast Answer:** Yes, **@st.cache_data(ttl=30)** caches event queries for 30 seconds.

Q20: What database driver library is used in Python?

- **Fast Answer:** PyMongo (**pymongo.MongoClient**) with **certifi** TLS support.

Category 3: Cybersecurity & SHA-256 Audit (Q21 – Q30)**Q21: How does your SHA-256 audit log work?**

- **Fast Answer:** Each event record is serialized and hashed together with the hash of the previous record.
- **Detailed Answer:** $H_i = \text{SHA256}(H_{i-1} + \text{JSON}(\text{Record}_i))$. This creates an unbreakably linked linear hash chain starting from "**GENESIS**".

Q22: Where is the root hash stored?

- **Fast Answer:** In **output/audit_root_hash.txt**.
- **Detailed Answer:** The root hash **441e0fcff3badcfe7705ce8004b5f0189642d3033ccc370ded52b8e0cb9dcb05** represents the final digest commitment.

Q23: How does the system detect if a record has been tampered with?

- **Fast Answer:** **verify_secure_audit_log()** recomputes all hashes step-by-step and flags any mismatch.
- **Detailed Answer:** If Record 2 is modified, its recomputed hash changes. Record 3's stored **previous_hash** won't match, causing the verification engine to fail and display a red alert.

Q24: What happens if someone edits both Record 2 AND recomputes all subsequent hashes?

- **Fast Answer:** The final computed hash will fail to match the immutable **audit_root_hash.txt**.

- **Detailed Answer:** Because `audit_root_hash.txt` can be backed up offsite or stored on a read-only HSM key, re-calculating downstream hashes still produces a root digest mismatch.

Q25: Why use SHA-256 instead of MD5 or SHA-1?

- **Fast Answer:** MD5 and SHA-1 have known collision vulnerabilities; SHA-256 is the cryptographic standard.

Q26: Can evidence images be tampered with on disk?

- **Fast Answer:** The evidence image filename and path are bound inside the hashed JSON record payload.

Q27: How fast is audit chain verification?

- **Fast Answer:** Near instantaneous ($O(N)$ linear time complexity for N records).

Q28: How is audit integrity presented to the user?

- **Fast Answer:** The Audit Log page displays a green ✓ **Audit Log Integrity VERIFIED** banner or a red error warning.

Q29: What is the previous hash for the very first event record?

- Fast Answer: "GENESIS".

Q30: Are audit records timestamped?

- **Fast Answer:** Yes, with frame number, video timestamp (**MM:SS.ms**), and system ISO creation time.

Category 4: Blockchain & Ledger (Q31 – Q40)

Q31: Is IBVAP using a real distributed blockchain network?

- **Fast Answer:** IBVAP uses a single-node local hash-linked audit ledger (**blockchain_ledger.json**) as a working proof-of-concept.
- **Detailed Answer:** We provide complete transparency: it is a block-linked JSON structure using SHA-256 block headers. Distributed P2P consensus is part of our future roadmap.

Q32: What is contained inside a block in `blockchain_ledger.json`?

- **Fast Answer:** `block_index`, `timestamp`, `data` (event dictionary), `previous_hash`, and `block_hash`.

Q33: How many blocks are currently generated in your project?

- **Fast Answer:** 4 blocks (Block 0 Genesis + Blocks 1, 2, 3 for EVT-001, EVT-002, EVT-003).

Q34: What is the previous hash of Block 0 (Genesis Block)?

- [illegible]

Q35: How is blockchain ledger verification executed?

- **Fast Answer:** `verify_blockchain_ledger()` recalculates block hashes and validates block link pointers.

Q36: Does your blockchain use Proof-of-Work (PoW)?

- **Fast Answer:** No, PoW is unnecessary for private defense audit logging due to high energy consumption.

Q37: What permissioned blockchain framework would you deploy to in production?

- **Fast Answer:** Hyperledger Fabric.

Q38: Why is a blockchain-style ledger better than a standard database table?

- **Fast Answer:** Standard DB tables allow **UPDATE** and **DELETE** commands; blockchain ledgers are append-only.

Q39: Where is `blockchain_ledger.json` created?

- **Fast Answer:** By `create_blockchain_ledger()` in `src/app.py`.

Q40: What is the block hash of Block 0?

- **Fast Answer:** `90bbb09388f2fba85c2b8261c1367bac2c27f92531ff74f1b370782cc150b9f5`.

Category 5: Deployment, UI & General (Q41 – Q50)

Q41: Where is IBVAP currently deployed?

- **Fast Answer:** Streamlit Community Cloud connected to GitHub.

Q42: What Python web framework powers the dashboard?

- **Fast Answer:** Streamlit (1.38+).

Q43: How many pages are in the Streamlit application?

- **Fast Answer:** 6 pages (Command Center, Surveillance, Security Events, Digital Evidence, Audit Log, System).

Q44: How are Plotly charts used in the Command Center?

- **Fast Answer:** For rendering Risk Level Distribution bar charts and Intrusion Frame scatter plots.

Q45: How do you handle UI theme switching?

- **Fast Answer:** Configured via `streamlit_app/.streamlit/config.toml` using custom CSS styling.

Q46: Can the dashboard be viewed on mobile devices?

- **Fast Answer:** Yes, CSS media queries (`@media (max-width: 768px)`) ensure responsive layout scaling.

Q47: What dependencies are required to run the project?

- **Fast Answer:** `streamlit`, `pandas`, `plotly`, `supabase-py`, `Pillow`, `requests`, `pymongo`, `certifi`, `ultralitics`, `opencv-python`.

Q48: How does the system handle large video files?

- **Fast Answer:** Videos are processed frame-by-frame on disk without loading entire uncompressed streams into RAM.

Q49: Is facial recognition implemented in IBVAP?

- **Fast Answer:** No. We track body targets via ByteTrack without storing biometric facial data, preserving privacy compliance.

Q50: What is the single biggest innovation in IBVAP?

- **Fast Answer:** The seamless combination of real-time YOLO11/ByteTrack vision with zero-downtime multi-tier data resilience and SHA-256 cryptographic audit integrity.